

English Game

A Next.js web application for learning English through interactive games about wireframes.

Tech Stack

- **Frontend / Backend:** Next.js 16 (React 19, App Router)
- **Database:** PostgreSQL 16
- **ORM:** Prisma
- **Auth:** better-auth + Google OAuth
- **HTTP Proxy:** Nginx

Architecture (Docker Compose)

The project runs with 3 containers:

Container	Role	Exposed?
postgres	PostgreSQL database	No (internal network only)
app	Next.js application (custom image)	No
nginx	Reverse proxy / HTTP server	Yes (port 80)

The database data is stored in a Docker volume (`pgdata`) so it persists across restarts.

Prerequisites

- [Docker](#) and Docker Compose installed
- (Optional) Google OAuth credentials if you want social login

Getting Started

1. Clone the repository

```
git clone https://github.com/tim-vdb/english-game.git
cd english-game
```

2. Configure environment variables

Copy the example env file and fill in your values:

```
cp .env.example .env
```

Edit `.env` and set at least:

- **BETTER_AUTH_SECRET** — any random string (used to sign auth tokens)
- **GOOGLE_CLIENT_ID** / **GOOGLE_CLIENT_SECRET** — only needed for Google login

The database credentials have sensible defaults for local development.

3. Build and start

```
docker compose up --build -d
```

This will:

1. Build the Next.js app image (custom Dockerfile)
2. Start PostgreSQL, the app, and Nginx
3. Automatically run Prisma migrations on first startup

The app will be available at **http://localhost**.

4. Check logs

```
# all services
docker compose logs -f

# just the app
docker compose logs -f app
```

5. Stop

```
docker compose down
```

Code Changes for Docker

The following modifications were made to the source code to support the Docker setup:

- **next.config.ts**: Added **output**: "standalone" for a self-contained production build.
- **Dockerfile**: Multi-stage build (build + runner) using **node:20-alpine**. Sets **NEXT_PUBLIC_API_URL=http://localhost** at build time so the auth client connects through Nginx. Runs Prisma migrations then starts the production server.
- **src/lib/auth.ts**: Added **trustedOrigins**: ["http://localhost", "http://localhost:3000", "http://localhost:80"] to allow authentication requests from the Nginx reverse proxy.
- **nginx/nginx.conf**: Reverse proxy configuration that forwards HTTP traffic to the app container on port 3000.

Development (without Docker)

If you want to run the app locally without Docker, you need Node.js 20+ and a PostgreSQL instance:

```
npm install
# set DATABASE_URL in your environment or .env file
npx prisma migrate dev --schema=src/prisma/schema.prisma
npm run dev
```